



EVALUATING DATA PARALLELISM IN C++ PROGRAMMING MODELS USING THE PARALLEL RESEARCH KERNELS

Jeff Hammond, Exascale Co-Design Group
Tim Mattson, Parallel Computing Lab
Intel Corporation

4 April 2019

Acknowledgements: **Rob van der Wijngaart**, Alex Duran, Jim Cownie, Alexey Kukanov, Pablo Reble, Xinmin Tian, Martyn Corden, Tom Scoglund and the rest of the RAJA team at LLNL, CodePlay SYCL team, ...

Abstract

Modern C++ provides a wide range of parallel constructs in the language itself, as well as tools to implement general and domain-specific parallel frameworks for both CPUs and accelerators. Examples include Threading Building Blocks (TBB), RAJA, Kokkos, HPX, Thrust, SYCL, and Boost.Compute, which complement the C++17 parallel STL.

This talk will describe our attempts to systematically compare these models against lower-level models like OpenMP and OpenCL. One goal is to understand the tradeoffs between performance, programmability and portability in these frameworks to educate HPC programmers.

The experiments are based on the Parallel Research Kernels (<https://github.com/ParRes/Kernels/>), which is a collection of application proxies associated with high-performance scientific computing applications such as partial differential equation solvers, deterministic neutron transport, 3D Fast Fourier Transforms, and dense linear algebra.

Notices and Disclaimers

© 2018 Intel Corporation. Intel, the Intel logo, Xeon and Xeon logos are trademarks of Intel Corporation in the U.S. and/or other countries

Intel technologies' features and benefits depend on system configuration and may require enabled hardware, software or service activation. . Performance varies depending on system configuration. **No computer system can be absolutely secure.** Check with your system manufacturer or retailer or learn more at intel.com/performance/datacenter.

Some results have been estimated or simulated using internal Intel analysis or architecture simulation or modeling, and provided to you for informational purposes. Any differences in your system hardware, software or configuration may affect your actual performance.

The cost reduction scenarios described are intended to enable you to get a better understanding of how the purchase of a given Intel based product, combined with a number of situation-specific variables, might affect future costs and savings. Circumstances will vary and there may be unaccounted-for costs related to the use and deployment of a given product. Nothing in this document should be interpreted as either a promise of or contract for a given level of costs or cost reduction.

Intel processors of the same SKU may vary in frequency or power as a result of natural variability in the production process.

All information provided here is subject to change without notice. Contact your Intel representative to obtain the latest Intel product specifications and roadmaps.

For more complete information about performance and benchmark results, visit www.intel.com/benchmarks.

Optimization Notice: Intel's compilers may or may not optimize to the same degree for non-Intel microprocessors for optimizations that are not unique to Intel microprocessors. These optimizations include SSE2, SSE3, and SSSE3 instruction sets and other optimizations. Intel does not guarantee the availability, functionality, or effectiveness of any optimization on microprocessors not manufactured by Intel. Microprocessor-dependent optimizations in this product are intended for use with Intel microprocessors. Certain optimizations not specific to Intel microarchitecture are reserved for Intel microprocessors. Please refer to the applicable product User and Reference Guides for more information regarding the specific instruction sets covered by this notice. Notice Revision #20110804.

Performance estimates were obtained prior to implementation of recent software patches and firmware updates intended to address exploits referred to as "Spectre" and "Meltdown." Implementation of these updates may make these results inapplicable to your device or system. Performance tests, such as SYSmark and MobileMark, are measured using specific computer systems, components, software, operations and functions. Any change to any of those factors may cause the results to vary. You should consult other information and performance tests to assist you in fully evaluating your contemplated purchases, including the performance of that product when combined with other products. For more information go to www.intel.com/benchmarks

*Other names and brands may be claimed as the property of others.

Additional Disclaimer

I am not an official spokesman for any Intel products. I do not speak for my collaborators, whether they be inside or outside Intel.

I work on system pathfinding and workload analysis, not software products. I am not a developer of Intel software tools.

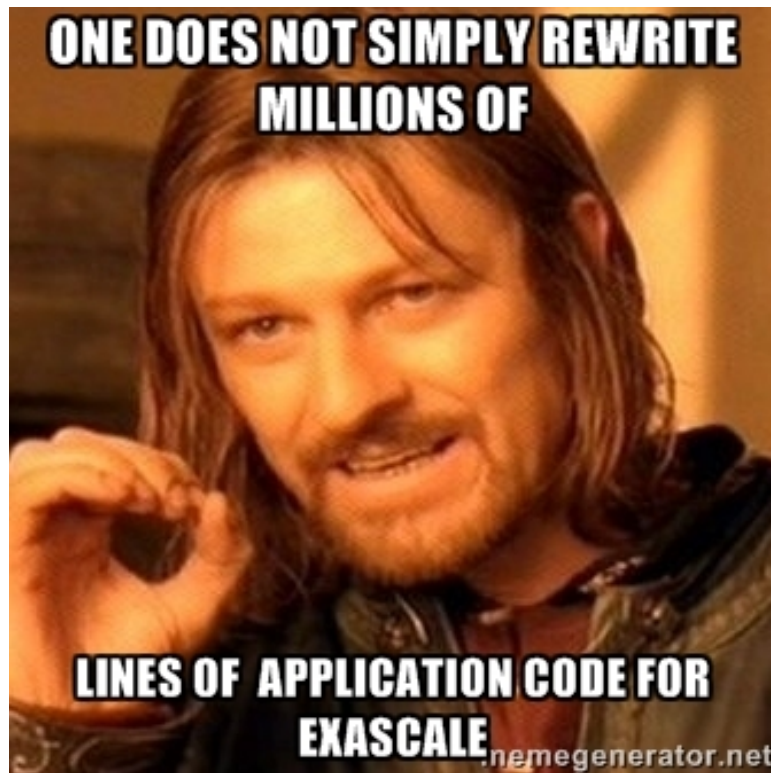
You may or may not be able to reproduce any performance numbers I report, but the code is on GitHub* and I will provide anything else you need to attempt to reproduce my results.

Hanlon's Razor (blame stupidity, not malice).

HPC software design challenges (2014)

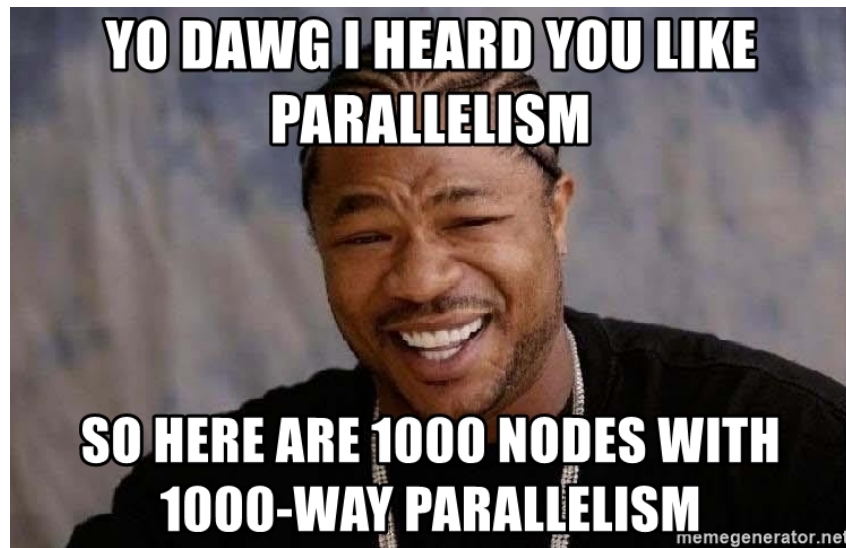
- To MPI or not to MPI...
- One-sided vs. two-sided?
- Does your MPI/PGAS need a +X?
- Static vs. dynamic execution model?
- What synchronization motifs maximize performance across scales?

Application programmers can afford to rewrite/redesign applications zero to one times every 20 years...



HPC software design challenges (2018)

- Intranode parallelism is growing much faster than internode...
- Intranode parallelism is far more diverse than internode parallelism.
- After ~20 years, internode behavior is converged to some subset of MPI-3.
- Big Cores, Little Cores, GPU, FPGA all require (very) different programming models.



How do we *measure* productivity+performance+portability?

PARALLEL RESEARCH KERNELS

Programming model evaluation

Standard methods:

- NAS Parallel Benchmarks
- Mini/Proxy Applications
- HPC Challenge

There are numerous examples of these on record, covering a wide range of programming models, but is source available and curated?

What is measured?

- Productivity (?), elegance (?)
- Implementation quality (runtime or application)
- Asynchrony/overlap
- Semantics:
 - Automatic load-balancing (AMR)
 - Atomics (GUPS)
 - Two-sided vs. one-sided, collectives

Benchmark Suite Scorecard (EMBRACE 2017)

	HPC user interest	clear paper & pencil	fast reference code	clear reference code	forum for comparison	productivity framework	prescribed approach	open competition	historical continuum	apples-to-apples	self-serve comparisons	self-run entries
NPB	✓	~	✓	~	✗	✗	✗					
HPCC	~	~	✓	✗	✓	✓	~	✓	✗	✗	✗	✗
DOE PROXY APPS	✓	~	✓	~	✗	✗	~					
CLBG	✗	✗	✓	✓	✓	✓	✓	~	✓	✓	✓	✓
PRK	~	✓	✓	?	~	✗	✓	✓				✓
	COMPUTE						ANALYZE					

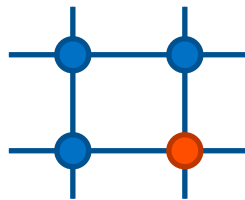
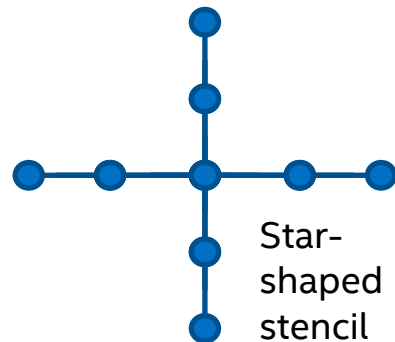
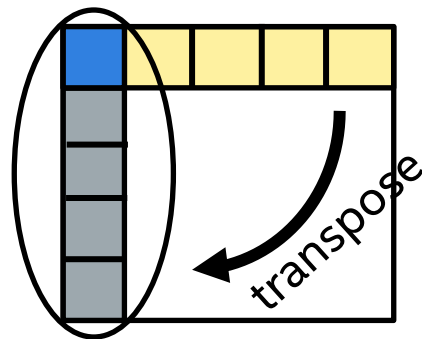
Copyright 2017 Cray Inc.

Goals of the Parallel Research Kernels

1. Universality: Cover broad range of performance critical application patterns.
2. Simplicity: Concise pencil-and-paper definition and transparent reference implementation. No domain knowledge required.
3. Portability: Should be implementable in any sufficiently general programming model.
4. Extensibility: Parameterized to run at any scale. Other knobs to adjust problem or algorithm included.
5. Verifiability: Automated correctness checking and built-in performance metric evaluation.
6. Hardware benchmark: No! Use HPCChallenge, Xyz500, etc. for this.

Outline of PRK Suite

- **Dense matrix transpose**
- Synchronization: global
- **Synchronization: point to point**
- **Scaled vector addition**
- Atomic reference counting
- Vector reduction
- Sparse matrix-vector multiplication
- Random access update
- **Stencil computation**
- Dense matrix-matrix multiplication
- Branch
- Particle-in-cell
- AMR



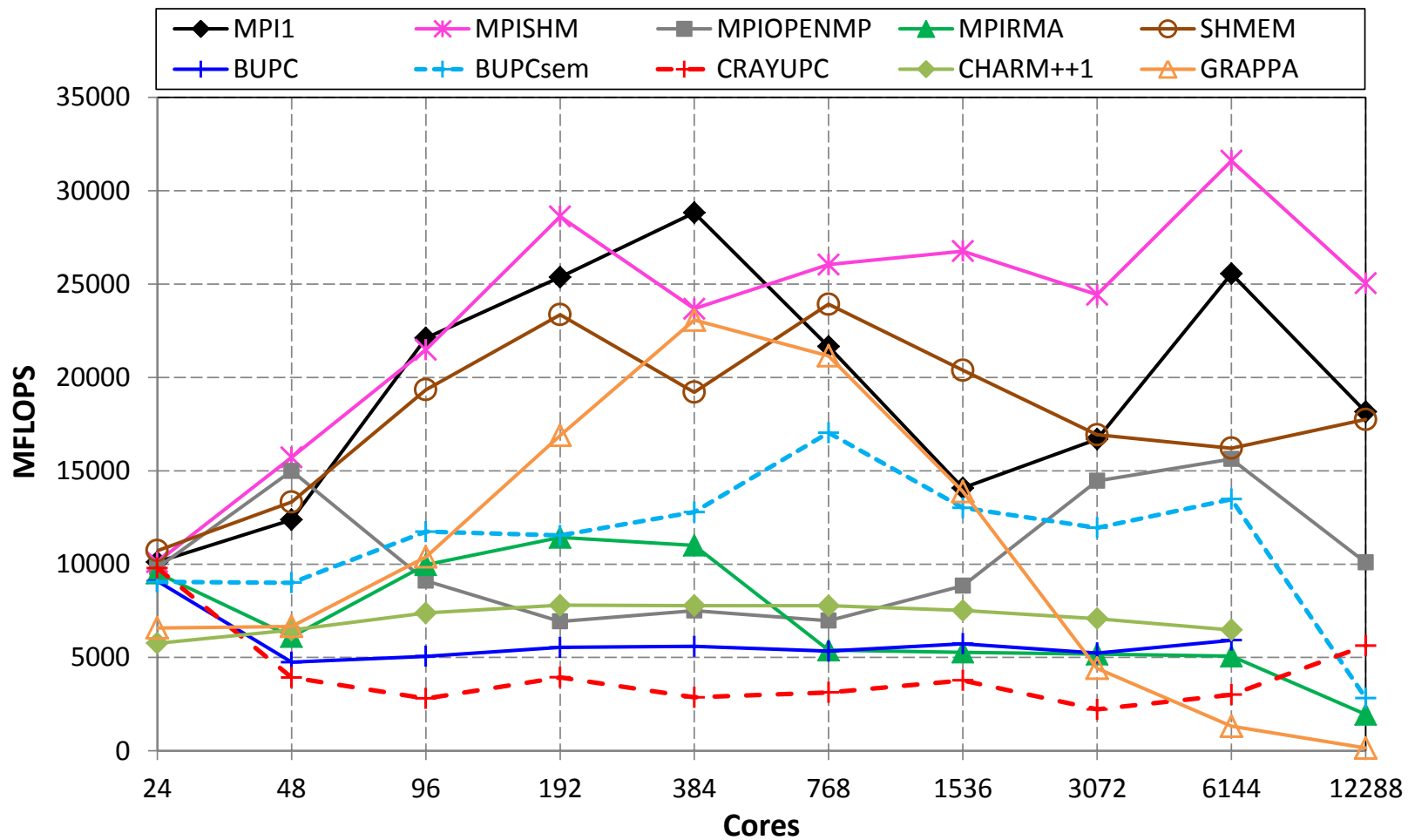
$$A_{i,j} = A_{i-1,j} + A_{i,j-1} - A_{i-1,j-1}$$

Static kernels

Language	Seq.	OpenMP	MPI	PGAS	Threads	Others?
C89	✓	✓	Many	SHMEM		
C99/C11	✓	✓✓✓		UPC	✓	Cilk, ISPC
C++17	✓	✓✓✓		Grappa	✓	Kokkos, RAJA, TBB, PSTL, SYCL, OpenCL, CUDA...
Fortran	✓	✓✓✓		coarrays		“pretty”, OpenACC
Python	✓					Numpy
Chapel	✓			✓		

✓✓✓ = Traditional, task-based, and target are implemented identically in Fortran, C and C++.

Additional language support includes Rust, Julia, and Matlab/Octave.



<> Code

🔔 Issues 27

🔗 Pull requests 3

📁 Projects 0

📖 Wiki

📊 Insights

⚙️ Settings

This is a set of simple programs that can be used to explore the features of a parallel platform. <https://groups.google.com/forum/#!for...> Edit

- parallel-programming
- parallel-computing
- c
- c-plus-plus
- mpi
- fortran2008
- python3
- julia
- pgas
- openmp
- shmem
- coarray-fortran
- travis-ci
- charmplusplus
- threading
- tbb
- kokkos
- opencl
- sycl
- boost

[Manage topics](#)

🕒 2,822 commits

🌿 11 branches

📦 6 releases

👤 19 contributors

📄 View license

Branch: master ▾

New pull request

Create new file

Upload files

Find File

Clone or download ▾

 jeffhammond	avoid overflow	Latest commit 30a2c6f 19 days ago
 .github	try out issue/PR templates	a year ago
 AMPI	avoid overflow	16 days ago
 C1z	OpenCL: add No Device errors (#373)	29 days ago
 CHARM++	avoid overflow	16 days ago
 Cxx11	do not incorrectly declare non-read-only buffers as read-only	19 days ago



✗ master CRON avoid overflow

#1410 failed

Restart build

Commit 30a2c6f

Branch master

Jeff Hammond authored and committed

Ran for 1 hr 44 min 57 sec

Total time 4 hrs 45 min 26 sec

6 days ago

Build jobs

View config

✓ # 1410.1	</> Compiler: gcc C++	PRK_TARGET=allserial	2 min 1 sec	
✓ # 1410.2	</> Compiler: clang C++	PRK_TARGET=allserial	5 min 33 sec	
✓ # 1410.3	</> Compiler: gcc C++	PRK_TARGET=allc1z	6 min 53 sec	
✓ # 1410.4	</> Compiler: clang C++	PRK_TARGET=allc1z	1 min 9 sec	
✓ # 1410.5	</> Compiler: gcc C++	PRK_TARGET=allcxx	8 min 13 sec	
✓ # 1410.6	</> Compiler: clang C++	PRK_TARGET=allcxx	6 min 35 sec	
✓ # 1410.7	</> Compiler: clang C++	PRK_TARGET=allpython	7 min 9 sec	
✓ # 1410.8	</> Compiler: clang C++	PRK_TARGET=alljulia	5 min 16 sec	
✓ # 1410.9	</> Compiler: gcc C++	PRK_TARGET=allopenmp	3 min 7 sec	
✓ # 1410.10	</> Compiler: gcc C++	PRK_TARGET=allfortran	14 min 44 sec	
✓ # 1410.11	</> Compiler: gcc C++	PRK_TARGET=allmpi	16 min 20 sec	
✓ # 1410.12	</> Compiler: gcc C++	PRK_TARGET=allshmem	4 min 59 sec	

✖ master CRON a

Commit 30a2c6

Branch master

Jeff Hammond aut

Bu

✓ # 1410.1

✓ # 1410.2

✓ # 1410.3

✓ # 1410.4

✓ # 1410.5

✓ # 1410.6

✓ # 1410.7

✓ # 1410.8

✓ # 1410.9

✓ # 1410.10

✓ # 1410.11

✓ # 1410.12

```
5154 +SYCLDIR=/Users/travis/build/ParRes/Kernels/PRK-deps/triSYCL
5155 +'[' clang = clang ']'
5156 +echo 'SYCLCXX=/usr/local/opt/llvm/bin/clang++ -pthread -std=c++17'
5157 +echo 'SYCLFLAG=-DUSE_SYCL -I/Users/travis/build/ParRes/Kernels/PRK-deps/triSYCL/include'
5158 +make -C Cxx11 p2p-hyperplane-sycl stencil-sycl transpose-sycl nstream-sycl
5159 /usr/local/opt/llvm/bin/clang++ -pthread -std=c++17 -DPRKVERSION="2.16" -DUSE_SYCL -
I/Users/travis/build/ParRes/Kernels/PRK-deps/triSYCL/include -DUSE_SYCL -DUSE_2D_INDEXING=0 -DUSE_RANGES_TS -
I/Users/travis/build/ParRes/Kernels/PRK-deps/range-v3/include -DUSE_RANGES p2p-hyperplane-sycl.cc -o p2p-hyperplane-sycl
5160 In file included from p2p-hyperplane-sycl.cc:62:
5161 In file included from /Users/travis/build/ParRes/Kernels/PRK-deps/triSYCL/include/CL/sycl.hpp:40:
5162 In file included from /Users/travis/build/ParRes/Kernels/PRK-deps/triSYCL/include/CL/sycl/buffer.hpp:27:
5163 In file included from /Users/travis/build/ParRes/Kernels/PRK-deps/triSYCL/include/CL/sycl/queue.hpp:30:
5164 /Users/travis/build/ParRes/Kernels/PRK-deps/triSYCL/include/CL/sycl/property_list.hpp:63:1: error: call to unavailable
member function 'value': introduced in macOS 10.14
5165 TRISYCL_PROPERTY_HAS_GET(queue, enable_profiling)
5166 ^~~~~~
5167 /Users/travis/build/ParRes/Kernels/PRK-deps/triSYCL/include/CL/sycl/property_list.hpp:60:22: note: expanded from macro
'TRISYCL_PROPERTY_HAS_GET'
5168     return prop_name.value();
5169         ~~~~~^~~~~~
5170 /usr/local/opt/llvm/bin/../include/c++/v1/optional:938:33: note: candidate function has been explicitly made unavailable
5171     constexpr value_type const& value() const&
5172         ^
5173 /usr/local/opt/llvm/bin/../include/c++/v1/optional:947:27: note: candidate function not viable: 'this' argument has type
'const std::optional<property::queue::enable_profiling>', but method is not marked const
5174     constexpr value_type& value() &
5175         ^
5176 /usr/local/opt/llvm/bin/../include/c++/v1/optional:956:28: note: candidate function not viable: 'this' argument has type
'const std::optional<property::queue::enable_profiling>', but method is not marked const
5177     constexpr value_type&& value() &&
5178         ^
5179 /usr/local/opt/llvm/bin/../include/c++/v1/optional:965:34: note: candidate function not viable: no known conversion from
'const optional<...>' to 'const optional<...>' for object argument
5180     constexpr value_type const&& value() const&&
5181         ^
5182 1 error generated.
5183 make: *** [p2p-hyperplane-sycl] Error 1
```

More options

Restart build

2 min 1 sec

5 min 33 sec

6 min 53 sec

1 min 9 sec

8 min 13 sec

6 min 35 sec

7 min 9 sec

5 min 16 sec

3 min 7 sec

14 min 44 sec

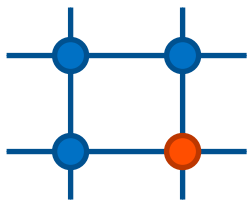
16 min 20 sec

Top 4 min 59 sec

Synch point-to-point

```
for i in range(1,m):  
    for j in range(1,n):  
        A[i][j] = A[i-1][j]  
                + A[i][j-1]  
                - A[i-1][j-1]
```

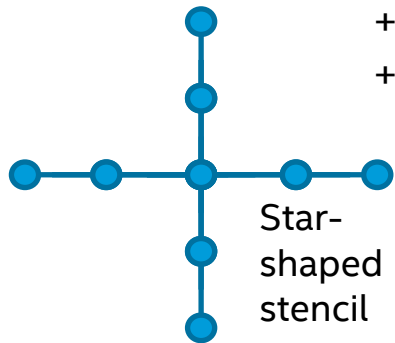
$$A[0][0] = -A[m-1][n-1]$$



$$A_{i,j} = A_{i-1,j} + A_{i,j-1} - A_{i-1,j-1}$$

- Proxy for discrete ordinates neutron transport; much simpler than SNAP or Kripke.
- Proxy for dynamic programming, which is used in sequence alignment (e.g. PairHMM).
- Wraparound to create dependency between iterations.

Stencil

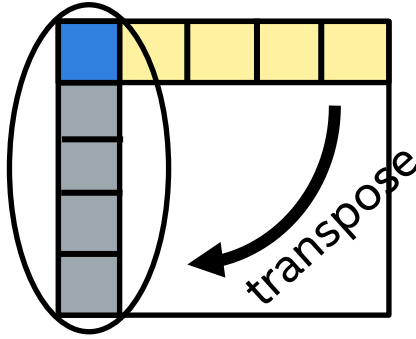
$$\begin{aligned} B[2:n-2,2:n-2] &+= W[2,2] * A[2:n-2,2:n-2] \\ &+ W[2,0] * A[2:n-2,0:n-4] \\ &+ W[2,1] * A[2:n-2,1:n-3] \\ &+ W[2,3] * A[2:n-2,3:n-1] \\ &+ W[2,4] * A[2:n-2,4:n-0] \\ &+ W[0,2] * A[0:n-4,2:n-2] \\ &+ W[1,2] * A[1:n-3,2:n-2] \\ &+ W[3,2] * A[3:n-1,2:n-2] \\ &+ W[4,2] * A[4:n-0,2:n-2] \end{aligned}$$


- Proxy for structured mesh codes. 2D stencil to emphasize non-compute.
- Supports arbitrary radius star and square stencils via code generator for C11 and C++ models, which was inspired by OpenCL.

Transpose

```
for i in range(order):  
    for j in range(order):  
        B[i][j] += A[j][i]  
        A[j][i] += 1.0
```

- Proxy for 3D FFT, bucket sort...
- Local transpose of square tiles supports blocking to reduce TLB pressure.



C++ AND PARALLELISM

I study molecular dynamics, but to tell the truth I am interested more in the dynamics than in the molecules, and I care most about questions of principle.

Phil Pechukas, Columbia University Chemical Physics Professor

I study C++ parallelism, but to tell the truth I am interested more in the parallelism than in the C++, and I care most about questions of practice.

Why C++ parallelism?

- C++ is a kitchen sink language – it has pretty much every feature that exists in programming languages (other than simplicity and orthogonality).
- Used across essentially all markets/domains where parallelism or performance matter.
 - Fortran and Rust usage domain-specific.
 - Interpreted languages do not satisfy performance requirements.
- C++ can be extended to do all sorts of things within the language itself. Variadic templates for fun and profit!
- Mattson's Law: No new languages!

Overview of Parallel C++ models

- TBB (Intel OSS) - parallel threading abstraction for CPU (+OpenCL kernels).
- KOKKOS (Sandia) – parallel execution and data abstraction for CPU and GPU architectures (OpenMP, Pthreads, CUDA, ...).
- RAJA (Livermore) – parallel execution for CPU and GPU architectures (OpenMP, TBB, CUDA, ...). CHAI adds GPU data abstraction.
- PSTL (ISO standard) – parallel execution abstraction for CPU architectures; designed for future extensions for GPU, etc. (e.g. Thrust and HPX).
- SYCL (Khronos standard) - parallel execution and data abstraction that extends the OpenCL model (supports CPU, GPU, FPGA, ...).

SYCL

- Khronos standard based on C++11 and OpenCL.
- Retains the OpenCL execution model: `work_groups` + `work_items`.
 - May require extensions for SIMD exec to support forward deps.
- Single-source programming model (may be >1 compiler passes).
- Eliminates the painful boilerplate code associated with OpenCL.
- OpenCL interoperability (e.g. OpenCL linear algebra libraries).

All experiments use the CodePlay* ComputeCpp implementation based on Clang/LLVM that generates SPIR-V.

Model	for	for ^N	reduce	scan	Hierarchy/Composition
TBB::parallel	Y	Y	Y	Y	Threads
C++17 PSTL	Y	N [^]	Y	Y	Threads+SIMD
RAJA	Y	Y	Y	Y	Threads+SIMD; CUDA
KOKKOS	Y	Y	Y	Y	Team+Thread+SIMD
Boost.Compute	Y	N ^{*^}	Y	Y	N
SYCL	Y	3	N	N	Group(+Subgroup)+Item
OpenCL	Y	3	N	N	Group+Item
OpenMP 5	Y	Y	Y	Y	Y ^{**}

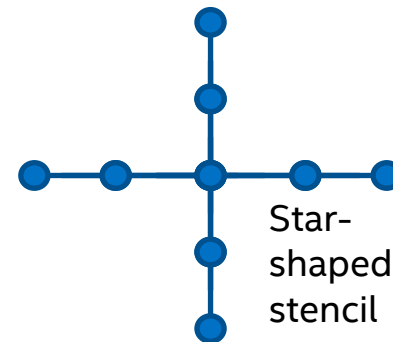
- * Boost.Compute supports embedded OpenCL, which in turn exposes 3D loop nests.
- ** OpenMP nested parallelism is unpleasant. You can nest “parallel for” or switch paradigms to “taskloop” and give up on accelerator support.
- ^ One can always implement a collapsed N-d loop but that adds div/mod to loop body.

HPC-like vs STL-like vs OpenCL-like

- TBB HPC-like
 - Nested, blocked forall w/ affinity control and load-balancing
- RAJA
 - Nested, blocked, permuted forall w/ fine-grain policy control.
- KOKKOS
 - Nested, blocked, permuted forall.

- C++17 (parallel STL) STL-like
 - Parallel STL evolving towards GPU etc.
- Boost.Compute
 - Effectively parallel STL over OpenCL.
- SYCL OpenCL-like
 - OpenCL execution model
 - Parallel STL over SYCL exists...

The HPC-like models capture the popular OpenMP idioms while hiding complexity.



PERFORMANCE EXPERIMENTS

<https://github.com/ParRes/Kernels/tree/master/Cxx11>

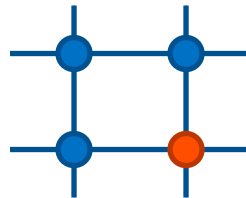
How to do performance experiments

1. Please download the latest version of the code from <https://github.com/ParRes/Kernels/>. The C++ stuff is in Cxx11/.
2. Look at `common/make.defs.<toolchain>` for a toolchain that your platform supports. We support all the platforms we can, hopefully including yours.
3. Copy `common/make.defs.<toolchain>` to `common/make.defs` and modify as necessary for your local environment. The file may contain useful comments.
4. Run your own performance experiments using good settings. For example, use OpenMP affinity properly.
5. If you have any problems, please file a GitHub issue or email Jeff Hammond.

WAVEFRONT PARALLELISM

Wavefront Parallelism

```
// sequential C implementation
for (int i=1; i<m; ++i) {
    for (int j=1; j<n; ++j) {
        A[i][j] = A[i-1][j] + A[i][j-1] - A[i-1][j-1];
    }
}
```

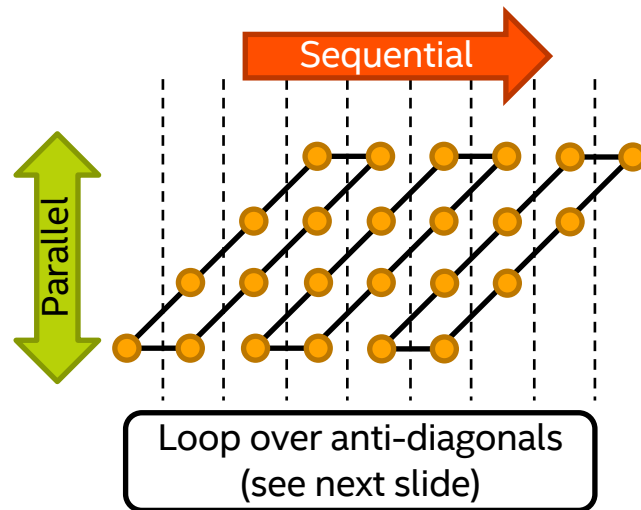
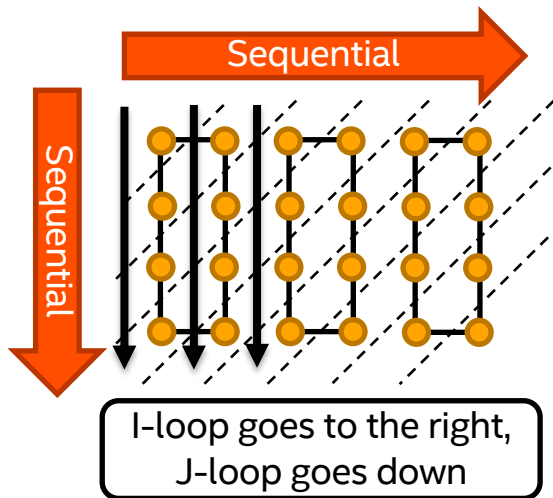


$$A_{i,j} = A_{i-1,j} + A_{i,j-1} - A_{i-1,j-1}$$

This *pattern* appears in a range of applications:

- Deterministic neutron transport (DOE-NNSA mission science)
- Smith-Waterman/PairHMM (bioinformatics)
- Dynamic programming
- Linear algebra (e.g. NAS LU benchmark)

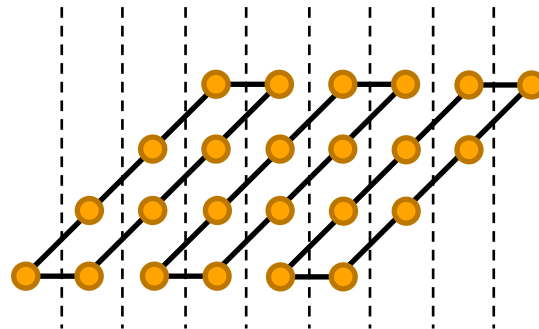
Changing the iteration space exposes parallelism



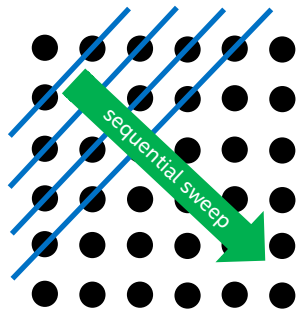
OpenMP inner-loop parallelism

```
// sequential loop
for (int i=2; i<=2*n-2; ++i) {
    int start = max(2,i-n+2);
    int stop  = min(i,n);
    #pragma omp for simd
    for (int j=start; j<=stop; ++j) {
        const int x = i-j+1;
        const int y = j-1;
        A[x][y] = A[x-1][y]
                  + A[x][y-1]
                  - A[x-1][y-1];
    }
    // implicit barrier (required)
}
```

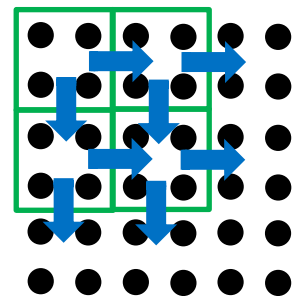
- Very low parallel efficiency once data spills private cache.
- CPU SIMD doesn't work because data access is non-contiguous.



Amortizing synchronization overheads



Parallel loop

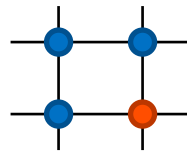


Task dependency

- Sequential execution requires no synchronization.
- Formally, there are $O(n^2)$ element-wise dependencies.
- Antidiagonal implementation uses $O(n)$ barriers to enforce deps.
- Hyperplane amortizes barriers across many antidiagonals: $O(n/\text{unroll})$ barriers.
- Task-based has $O(n^2/\text{block}^2)$ dependencies.

OpenMP task-based parallelism

```
#pragma omp parallel
#pragma omp master
for (int i=1; i<m; i+=mc) {
    for (int j=1; j<n; j+=nc) {
        #pragma omp task depend(in:grid[i-mc][j],grid[i][j-nc]) \
                               depend(out:grid[i][j])
        for (int ii=i; ii<std::min(m,i+mc); ii++) {
            for (int jj=j; jj<std::min(n,j+nc); jj++) {
                A[ii][jj] = A[ii-1][jj] + A[ii][jj-1] - A[ii-1][jj-1];
            }
        }
    }
}
#pragma omp taskwait
```

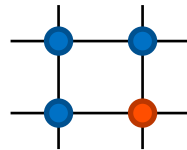


$$A_{i,j} = A_{i-1,j} + A_{i,j-1} - A_{i-1,j-1}$$

OpenMP “doacross” parallelism

The Intel OpenMP already has an improved implementation of this feature...

```
#pragma omp for collapse(2) ordered(2)
for (int i=0; i<ib; i++) {
    for (int j=0; j<jb; j++) {
        #pragma omp ordered depend(sink: i-1,j) depend(sink: i,j-1)
        for (int ii=i; ii<std::min(m,i+mc); ii++) {
            for (int jj=j; jj<std::min(n,j+nc); jj++) {
                A[ii][jj] = A[ii-1][jj] + A[ii][jj-1] - A[ii-1][jj-1];
            }
        }
        #pragma omp depend(source)
    }
}
```



$$A_{i,j} = A_{i-1,j} + A_{i,j-1} - A_{i-1,j-1}$$

OpenMP hyperplane parallelism

```
#pragma omp parallel
for (int i=2; i<=2*(nb+1)-2; i++) {
    #pragma omp for
    for (int j=std::max(2,i-(nb+1)+2); j<=std::min(i,nb+1); j++) {
        const int ib = nc*(i-j)+1;
        const int jb = nc*(j-2)+1;
        for (int ii=ib; ii<std::min(m,ib+nc); ii++) {
            for (int jj=jb; jj<std::min(n,jb+nc); jj++) {
                A[ii][jj] = A[ii-1][jj] + A[ii][jj-1] - A[ii-1][jj-1];
            }
        }
    }
}
```

This is only implemented for square grids to keep the polyhedral arithmetic simpler.

Summary

- Parallel C++ effectively hides the complexity of underlying models like OpenMP and OpenCL without introducing any overhead (on CPUs).
- Implementation differences between OpenMP and TBB schedulers show places where OpenMP runtimes can be improved.
- PSTL (based on TBB in Intel's implementation) works well on CPUs but is limited by STL semantics. PSTL portability requires evolution of C++ towards HPX, Thrust...
- SYCL provides a modern C++ abstraction and single-source compilation on top the OpenCL execution model.
- Task-based parallelism has a good ROI for wavefront algorithms.

Where do we go next?

- Rewrite RAJA code again (contributions welcome 😊); test wavefront policy.
- Evaluate performance on other platforms, particularly non-CPU ones.
- Performance optimization, particularly in stencil – how productive is tuning in different models?
- Write additional kernels:
 - Branch 2.0 (orient towards lane divergence, not branch predictor)
 - Reduce (different patterns, variable implementation quality)
- Julia vs Python vs Octave doesn't matter to me but others care.

References

- R. F. Van der Wijngaart, A. Kayi, J. R. Hammond, G. Jost, T. St. John, S. Sridharan, T. G. Mattson, J. Abercrombie, and J. Nelson. ISC 2016. *Comparing runtime systems with exascale ambitions using the Parallel Research Kernels.*
- E. Georganas, R. F. Van der Wijngaart and T. G. Mattson. IPDPS 2016. *Design and Implementation of a Parallel Research Kernel for Assessing Dynamic Load-Balancing Capabilities.*
- R. F. Van der Wijngaart and T. G. Mattson. HPEC 2014. *The Parallel Research Kernels.*

